

多媒體HW1_B1229062_林冠妤

實作方法:

- step 1:打開vs code, 下載更新python到python 3.10+(原本用的版本太舊了)

指令: `pip install numpy matplotlib scipy`

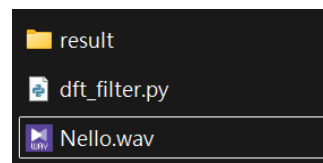
```
Downloading pillow-12.0.0-cp313-cp313-win_amd64.whl.metadata (9.0 kB)
Collecting pyarsing>=3 (from matplotlib)
  Downloading pyarsing-3.2.5-py3-none-any.whl.metadata (5.0 kB)
Collecting python-dateutil>=2.7 (from matplotlib)
  Downloading python_dateutil-2.9.0.post0-py2.py3-noneCollecting packaging>=20.0 (from matplotlib)
  Downloading matplotlib-3.10.7-cp313-cp313-win_amd64.whl (8.1 MB)
  8.1/8.1 MB 569.1 kB/s eta 0:00:00
Downloading scipy-1.16.3-cp313-cp313-win_amd64.whl (38.5 MB)
  38.5/38.5 MB 748.6 kB/s eta 0:00:00
Downloading contourpy-1.3.3-cp313-cp313-win_amd64.whl (226 kB)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.60.1-cp313-cp313-win_amd64.whl (2.3 MB)
  2.3/2.3 MB 1.1 MB/s eta 0:00:00
Downloading kiwisolver-1.4.9-cp313-cp313-win_amd64.whl (73 kB)
Downloading packaging-25.0-py3-none-any.whl (111 kB)
Downloading pillow-12.0.0-cp313-cp313-win_amd64.whl (7.0 MB)
  7.0/7.0 MB 1.1 MB/s eta 0:00:00
Downloading pyarsing-3.2.5-py3-none-any.whl (111 kB)
Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: six, pyarsing, pillow, packaging, numpy, kiwisolver, fonttools, cycler, scipy, python-dateutil, contourpy, m
matplotlib
Successfully installed contourpy-1.3.3 cycler-0.12.1 fonttools-4.60.1 kiwisolver-1.4.9 matplotlib-3.10.7 numpy-2.3.4 packaging-25.0 pillow-1
2.0.0 pyarsing-3.2.5 python-dateutil-2.9.0.post0 scipy-1.16.3 six-1.17.0

[notice] A new release of pip is available: 25.0.1 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\User\Desktop\DFT_Filter_HW1>
```

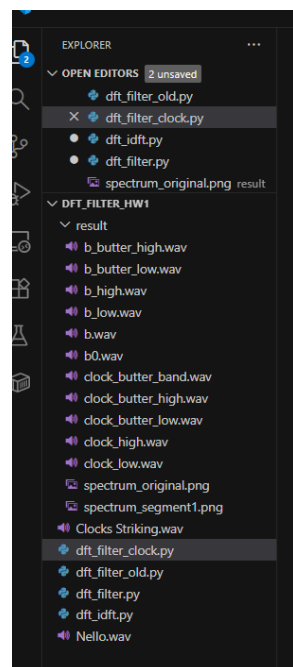
- step 2:建立資料夾, 將教授提供的Nello.wav檔放進去

資料夾名稱:DFT_Filter_HW1;

初始檔案建立:(result放輸出的輸出檔和圖表;.py檔放python程式)



最終檔案會有的:



- step 3:開始實作作業

工作進度一_作業題目0:前導練習

code使用:

- (1) 讀出 a.wav 並寫入 b.wav:使用 rb / wb 模式
- (2) 使用 buffer(二進位處理):使用 .read() 讀取 bytes
- (3) 檔名、輸出路徑正確:Nello.wav -> result/b.wav

程式碼(有附寫法註解) + 結果測試

```
dft_filter.py x dft_idft.py • spectrum_segment1.png
dft_filter.py > ...
1 # Homework 1: 前導練習_讀取Nello.wav並寫出b.wav
2 import os
3 # 原始檔名與目標檔名
4 input_file = "Nello.wav"
5 output_file = "result/b0.wav"
6 # 開啟原始檔(rb = read binary)
7 with open(input_file, "rb") as f_in:
8     data = f_in.read() # 讀取整個音檔內容(bytes)
9 # 開啟輸出檔(wb = write binary)
10 with open(output_file, "wb") as f_out:
11     f_out.write(data) # 把剛剛讀進來的內容原封不動地寫出
12 print("已輸出result/b0.wav(內容與Nello.wav相同)")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\user\Desktop\DFT_Filter_HW1> & C:/Users/user/AppData/Local/Programs/Python/Python313/python.exe c:/Users/user/Desktop/DFT_Filter_HW1/dft_filter.py
已輸出result/b0.wav(內容與Nello.wav相同)
PS C:\Users\user\Desktop\DFT_Filter_HW1> █
```

結果:輸出的音檔b0.wav會聽起來和原檔Nello.wav一樣。

工作進度二_作業題目1:離散傅立葉正逆轉換

- code使用:

實作離散傅立葉轉換(DFT)與逆轉換(IDFT), 驗證傅立葉分析與合成的可逆性。程式會以8bit單聲道音檔 Nello.wav 作為輸入, 會先將音訊訊號分段(這裡每段長度N=128), 再逐段進行DFT計算得到頻域複數資料 $A[k]+jB[k]$, 且未對頻譜做任何修改, 會直接以相同頻譜進行IDFT的還原, 最後才將結果重新組合並輸出成 b.wav, 理論上與原始音訊波形會是一樣的, 然後程式還有繪出第一段信號的頻譜圖 $(A^2[k]+B^2[k])^{(1/2)}$ 以驗證DFT的正確性。

程式碼(有附寫法註解)

終端機顯示的結果 + 結果測試

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

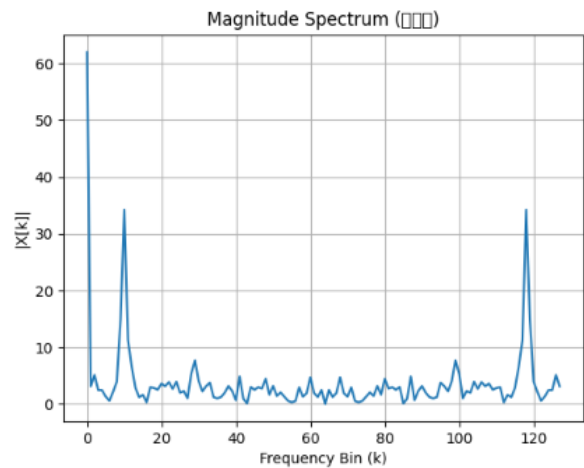
PS C:\Users\user\Desktop\DFT_Filter_HW1> & C:/Users/user/AppData/Local/Programs/Python/Python313/python.exe c:/Users/user/Desktop/DFT_Filter_HW1/dft_idft.py
總長度: 1362384 分塊長度: 128 共 10643 段
處理第 0 段 / 共 10643 段
C:\Users\user\Desktop\DFT_Filter_HW1\dft_idft.py:93: UserWarning: Glyph 31532 (WICK UNIFIED IDEOGRAPH-782C) missing from font(s) DejaVu Sans.
plt.savefig("result/spectrum_segment1.png")
C:\Users\user\Desktop\DFT_Filter_HW1\dft_idft.py:93: UserWarning: Glyph 19968 (WICK UNIFIED IDEOGRAPH-4E80) missing from font(s) DejaVu Sans.
plt.savefig("result/spectrum_segment1.png")
C:\Users\user\Desktop\DFT_Filter_HW1\dft_idft.py:93: UserWarning: Glyph 27573 (WICK UNIFIED IDEOGRAPH-68B5) missing from font(s) DejaVu Sans.
plt.savefig("result/spectrum_segment1.png")
繪制結果圖譜: result\spectrum_segment1.png
處理第 20 段 / 共 10643 段
處理第 40 段 / 共 10643 段
處理第 60 段 / 共 10643 段
處理第 80 段 / 共 10643 段
處理第 100 段 / 共 10643 段
處理第 120 段 / 共 10643 段
處理第 140 段 / 共 10643 段
處理第 160 段 / 共 10643 段
處理第 180 段 / 共 10643 段
處理第 200 段 / 共 10643 段
處理第 220 段 / 共 10643 段
處理第 240 段 / 共 10643 段
處理第 260 段 / 共 10643 段
處理第 280 段 / 共 10643 段
處理第 300 段 / 共 10643 段
處理第 320 段 / 共 10643 段
處理第 340 段 / 共 10643 段
處理第 10540 段 / 共 10643 段
處理第 10560 段 / 共 10643 段
處理第 10580 段 / 共 10643 段
處理第 10600 段 / 共 10643 段
處理第 10620 段 / 共 10643 段
處理第 10640 段 / 共 10643 段
處理第 10480 段 / 共 10643 段
處理第 10500 段 / 共 10643 段
處理第 10520 段 / 共 10643 段
處理第 10540 段 / 共 10643 段
處理第 10560 段 / 共 10643 段
處理第 10580 段 / 共 10643 段
處理第 10600 段 / 共 10643 段
處理第 10620 段 / 共 10643 段
處理第 10640 段 / 共 10643 段
處理第 10560 段 / 共 10643 段
處理第 10580 段 / 共 10643 段
處理第 10600 段 / 共 10643 段
處理第 10620 段 / 共 10643 段
處理第 10640 段 / 共 10643 段
完成輸出 result/b.wav
PS C:\Users\user\Desktop\DFT_Filter_HW1> █
```

產出的spectrum_segment1.png

```

dft_idft.py > ...
1 # HW1_離散傅立葉正逆轉換
2 # 題目：對Nello.wav做DFT->IDFT，不改變內容再輸出b.wav
3 # 限制：不得使用numpy.fft，只能自行撰寫DFT/IDFT
4 import wave # 讀寫 wav 檔案
5 import numpy as np # 處理陣列與數學運算
6 import os
7 import matplotlib.pyplot as plt # 畫頻譜圖
8 # 離散傅立葉轉換DFT(for loop版本)
9 def DFT(x):
10     N = len(x) # 資料長度
11     X = [] # 建立空陣列
12     for k in range(N): # 逐一計算每個頻率分量
13         real = 0.0
14         imag = 0.0
15         for n in range(N):
16             angle = 2 * np.pi * k * n / N
17             real += x[n] * np.cos(-angle) # 實部
18             imag += x[n] * np.sin(-angle) # 虛部
19         X.append(complex(real, imag)) # 寫入複數
20     return np.array(X)
21 # 離散傅立葉逆轉換IDFT(一樣是for loop版)
22 def IDFT(X):
23     N = len(X)
24     x = []
25     for n in range(N): # 對時間點n逐一計算
26         s = 0
27         for k in range(N): # 對所有頻率分量加總
28             angle = 2 * np.pi * k * n / N
29             s += X[k] * complex(np.cos(angle), np.sin(angle))
30         x.append((s / N).real) # 取實部
31     return np.array(x)
32 # 主程式
33 # Step(1)建立輸出資料夾
34 if not os.path.exists("result"):
35     os.makedirs("result")
36 # Step(2)讀取原始音檔
37 wav_in = wave.open("Nello.wav", "rb")
38 params = wav_in.getparams()
39 frames = wav_in.readframes(wav_in.getnframes())
40 wav_in.close()
41 # Step(3)將8bit unsigned轉成-128~127
42 data = np.frombuffer(frames, dtype=np.uint8)
43 data = data.astype(np.int16) - 128
44 # Step(4)設定分段長度(可改 128 / 256 / 512)

```



結果:輸出的音檔b0.wav會聽起來和原檔Nello.wav很像(聽起來一樣)。

```

43 data = data.astype(np.int16) - 128
44 # Step(4)設定分段長度(可改 128 / 256 / 512)
45 N = 128
46 if len(data) % N != 0:
47     pad = N - (len(data) % N)
48     data = np.append(data, np.zeros(pad))
49 segments = len(data) // N
50 result = np.zeros_like(data, dtype=float)
51 print("總長度:", len(data), "分段長度:", N, "共", segments, "段")
52 # Step(5)分段做 DFT -> IDFT
53 for s in range(segments):
54     start = s * N
55     end = start + N
56     segment = data[start:end]
57     #顯示進度
58     if s % 20 == 0:
59         print("處理第", s, "段 / 共", segments, "段")
60     #DFT
61     X = DFT(segment)
62     #第一段畫頻譜
63     if s == 0:
64         mag = np.sqrt(np.real(X)**2 + np.imag(X)**2)
65         plt.plot(mag)
66         plt.title("Magnitude Spectrum (第一段)")
67         plt.xlabel("Frequency Bin (k)")
68         plt.ylabel("|X[k]|")
69         plt.grid(True)
70         plt.savefig("result/spectrum_segment1.png")
71         plt.close()
72         print("已輸出頻譜圖 result/spectrum_segment1.png")
73     # 不進行濾波(原封不動)
74     Y = X
75     #IDFT
76     y = IDFT(Y)
77     result[start:end] = y
78 # Step(6)轉回8bit unsigned(0~255)
79 result = np.clip(result + 128, 0, 255).astype(np.uint8)
80 # Step(7)寫出音檔b.wav
81 wav_out = wave.open("result/b.wav", "wb")
82 wav_out.setparams(params)
83 wav_out.writeframes(result.tobytes())
84 wav_out.close()
85 print("完成輸出 result/b.wav")
86

```

- 參考來源說明：

本程式依據課堂投影片 HW1_DFT_Filter_2025.ppt 之作業要求自行撰寫。

DFT與IDFT的數學公式與推導依據簡報內容

以及課堂講解的離散傅立葉轉換定義式：

$$X[k] = \sum_{n=0}^{N-1} x[n] * e^{(-j2\pi kn/N)}$$

$$x[n] = (1/N) \sum_{k=0}^{N-1} X[k] * e^{(j2\pi kn/N)}$$

程式中之 DFT、IDFT 函式均由學生自行以 for 迴圈撰寫，並未使用任何現成轉換函數(如 numpy.fft)

其他部分(音檔讀寫、繪圖)參考Python官方文件：

<https://docs.python.org/3/library/wave.html>

<https://matplotlib.org/stable/index.html>

協助與偵錯:程式邏輯以及撰寫皆由本人先行完成，再使用ChatGPT協助檢查錯誤。

工作進度三_作業題目2:頻域的變音處理

- code使用：

用8bit單聲道音訊Nello.wav作為輸入，進行分段DFT分析、頻域濾波及IDFT重建，程式設計出四種濾波器：理想低通 (ILPF)、理想高通 (IHPF)、Butterworth 低通 (BLPF)與Butterworth 高通 (BHPF)，每種濾波器均使用自行撰寫的 DFT / IDFT 函式，未使用 numpy.fft，程式還有額外輸出四種濾波結果音檔與原始頻譜圖，可用以比較不同濾波器對音質的影響。

程式碼(有附寫法註解)

終端機顯示的結果 + 結果測試

```

dft_filter.py > DFT
1 # Hw1_項域的音響處理 (30%)
2 # 題目: 對hello.wav做DFT->filtering->IDFT
3 # 製作理想低通(BLPF)、理想高通(BHPF)、
4 # Butterworth低通(BLPF)、Butterworth高通(BHPF)
5 # 輸出四個音檔
6 # 限制: 不得使用numpy.fft, 只能自行撰寫DFT/IDFT
7
8 import numpy as np
9 import os
10 import matplotlib.pyplot as plt
11 # 自訂: 離散傅立葉轉換 DFT
12 def DFT(x):
13     N = len(x)
14     X = []
15     for k in range(N):
16         real = 0.0
17         imag = 0.0
18         for n in range(N):
19             angle = 2 * np.pi * k * n / N
20             real += x[n] * np.cos(angle)
21             imag += x[n] * np.sin(-angle)
22         X.append(complex(real, imag))
23     return np.array(X)
24 # 自訂: 離散傅立葉逆轉換 IDFT
25 def IDFT(X):
26     N = len(X)
27     x = []
28     for n in range(N):
29         s = 0
30         for k in range(N):
31             angle = 2 * np.pi * k * n / N
32             s += X[k] * complex(np.cos(angle), np.sin(angle))
33         x.append((s / N).real)
34     return np.array(x)
35
36 # 濾波器設計
37 def ideal_lowpass(X, cutoff):
38     """理想低通濾波器: 保留低頻、去除高頻"""
39     N = len(X)
40     H = np.zeros(N)
41     H[:cutoff] = 1
42     return X * H
43
44 def ideal_highpass(X, cutoff):
45     """理想高通濾波器: 保留高頻、去除低頻"""

```

```

45     N = len(X)
46     H = np.ones(N)
47     H[:cutoff] = 0
48     H[-cutoff:] = 0
49     return X * H
50
51 def butterworth_lowpass(X, cutoff, n=2):
52     """Butterworth 低通濾波器 (BLPF)"""
53     N = len(X)
54     H = np.zeros(N)
55     for k in range(N):
56         D = abs(k - N / 2) # 距離中心頻率的距離
57         H[k] = 1 / (1 + (D / cutoff) ** (2 * n))
58     return X * H
59
60 def butterworth_highpass(X, cutoff, n=2):
61     """Butterworth 高通濾波器 (BHPF)"""
62     N = len(X)
63     H = np.zeros(N)
64     for k in range(N):
65         D = abs(k - N / 2)
66         # 高通濾波器公式為 1 / (1 + (cutoff / D)^(2n))
67         if D == 0: # 避免除以 0
68             H[k] = 0
69         else:
70             H[k] = 1 / (1 + (cutoff / D) ** (2 * n))
71     return X * H
72
73 # 主程式開始
74 # Step(1)建立輸出資料夾
75 if not os.path.exists("result"):
76     os.makedirs("result")
77 # Step(2)讀取Nello.wav音檔
78 wav_in = wave.open("Nello.wav", "rb")
79 params = wav_in.getparams()
80 frames = wav_in.readframes(wav_in.getnframes())
81 wav_in.close()
82 # Step(3)將8001 unsigned轉成-128~127 的整數
83 data = np.frombuffer(frames, dtype=np.int8)
84 data = data.astype(np.int16) - 128
85 # Step(4)設定分段長度N(可改 128 / 256 / 512)
86 N = 256
87 if len(data) % N != 0:
88     pad = N - (len(data) % N)
89     data = np.append(data, np.zeros(pad))
90
91 data = data[:len(data)]

```

```

PS C:\Users\user\Desktop\DFT_Filter_HM1> & C:\Users\user\AppData\Local\Program\Python\Python311\python.exe c:/Users/user/Desktop/DFT_Filter_HM1/dft_filter.py
資料長度: 1362432 每段 N = 256 共 5322 段
進度: 0 / 5322
c:\Users\user\Desktop\DFT_Filter_HM1\dft_filter.py:140: UserWarning: Glyph 21407 (U+530F) missing from font(s) DejaVu Sans.
plt.savefig("result/spectrum_original.png")
c:\Users\user\Desktop\DFT_Filter_HM1\dft_filter.py:140: UserWarning: Glyph 22987 (U+530F) missing from font(s) DejaVu Sans.
plt.savefig("result/spectrum_original.png")
c:\Users\user\Desktop\DFT_Filter_HM1\dft_filter.py:140: UserWarning: Glyph 38971 (U+530F) missing from font(s) DejaVu Sans.
plt.savefig("result/spectrum_original.png")
c:\Users\user\Desktop\DFT_Filter_HM1\dft_filter.py:140: UserWarning: Glyph 19968 (U+530F) missing from font(s) DejaVu Sans.
plt.savefig("result/spectrum_original.png")
c:\Users\user\Desktop\DFT_Filter_HM1\dft_filter.py:140: UserWarning: Glyph 27573 (U+530F) missing from font(s) DejaVu Sans.
已輸出四個音檔: result/spectrum_original.png
進度: 100 / 5322
進度: 200 / 5322
進度: 300 / 5322
進度: 400 / 5322
進度: 500 / 5322
進度: 600 / 5322
進度: 700 / 5322
進度: 800 / 5322
進度: 900 / 5322
進度: 1000 / 5322
進度: 1100 / 5322
進度: 1200 / 5322
進度: 1300 / 5322
進度: 1400 / 5322
進度: 1500 / 5322
進度: 1600 / 5322
進度: 1700 / 5322
進度: 1800 / 5322
進度: 1900 / 5322
進度: 2000 / 5322
進度: 2100 / 5322
進度: 2200 / 5322
進度: 2300 / 5322
進度: 2400 / 5322
進度: 2500 / 5322
進度: 2600 / 5322
進度: 2700 / 5322
進度: 2800 / 5322
進度: 2900 / 5322
進度: 3000 / 5322
進度: 3100 / 5322
進度: 3200 / 5322
進度: 3300 / 5322
進度: 3400 / 5322
進度: 3500 / 5322
進度: 3600 / 5322
進度: 3700 / 5322
進度: 3800 / 5322
進度: 3900 / 5322
進度: 4000 / 5322
進度: 4100 / 5322
進度: 4200 / 5322
進度: 4300 / 5322
進度: 4400 / 5322
進度: 4500 / 5322
進度: 4600 / 5322
進度: 4700 / 5322
進度: 4800 / 5322
進度: 4900 / 5322
進度: 5000 / 5322
進度: 5100 / 5322
進度: 5200 / 5322
進度: 5300 / 5322
已輸出四個音檔:
result/b_low.wav -> 理想低通
result/b_high.wav -> 理想高通
result/b_butter_low.wav -> Butterworth 低通
進度: 5100 / 5322
進度: 5200 / 5322
進度: 5300 / 5322
已輸出四個音檔:
result/b_low.wav -> 理想低通
result/b_high.wav -> 理想高通
result/b_butter_low.wav -> Butterworth 低通
result/b_butter_high.wav -> Butterworth 高通
PS C:\Users\user\Desktop\DFT_Filter_HM1>

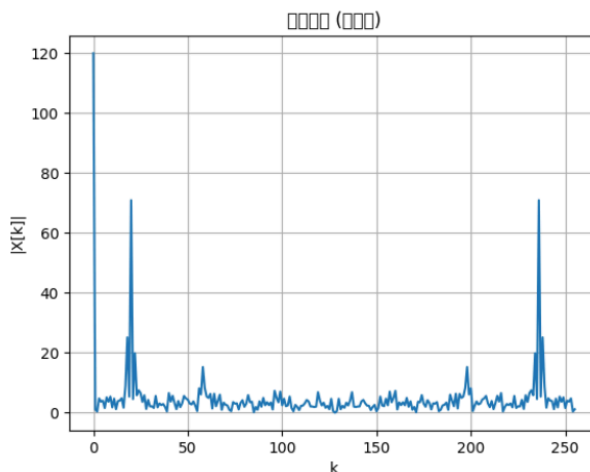
```

```

進度: 2400 / 5322
進度: 2500 / 5322
進度: 2600 / 5322
進度: 2700 / 5322
進度: 2800 / 5322
進度: 2900 / 5322
進度: 3000 / 5322
進度: 3100 / 5322
進度: 3200 / 5322
進度: 3300 / 5322
進度: 3400 / 5322
進度: 3500 / 5322
進度: 3600 / 5322
進度: 3700 / 5322
進度: 3800 / 5322
進度: 3900 / 5322
進度: 4000 / 5322
進度: 4100 / 5322
進度: 4200 / 5322
進度: 4300 / 5322
進度: 4400 / 5322
進度: 4500 / 5322
進度: 4600 / 5322
進度: 4700 / 5322
進度: 4800 / 5322
進度: 4900 / 5322
進度: 5000 / 5322
進度: 5100 / 5322
進度: 5200 / 5322
進度: 5300 / 5322
已輸出四個音檔:
result/b_low.wav -> 理想低通
result/b_high.wav -> 理想高通
result/b_butter_low.wav -> Butterworth 低通
進度: 5100 / 5322
進度: 5200 / 5322
進度: 5300 / 5322
已輸出四個音檔:
result/b_low.wav -> 理想低通
result/b_high.wav -> 理想高通
result/b_butter_low.wav -> Butterworth 低通
result/b_butter_high.wav -> Butterworth 高通
PS C:\Users\user\Desktop\DFT_Filter_HM1>

```

spectrum_original.png



結果:

```

87 segments = len(data) // N
88 print("資料長度:", len(data), "每段 N =", N, "共", segments, "段")
89 # Step(5)建立四個輸出陣列
90 out_low = np.zeros_like(data, dtype=float)
91 out_high = np.zeros_like(data, dtype=float)
92 out_butter_low = np.zeros_like(data, dtype=float)
93 out_butter_high = np.zeros_like(data, dtype=float)
94 # Step(6)設定截止頻率(cutoff)
95 cutoff = 40
96 # Step(7)每段做 DFT -> 濾波 -> IDFT
97 for s in range(segments):
98     start = s * N
99     end = start + N
100     seg = data[start:end]
101     if s % 100 == 0:
102         print("進度:", s, "%")
103     X = DFT(seg)
104     # 第一段畫頻譜
105     if s == 0:
106         mag = np.sqrt(np.real(X)**2 + np.imag(X)**2)
107         plt.figure()
108         plt.plot(mag)
109         plt.title("原始頻譜 (第一段)")
110         plt.xlabel("k")
111         plt.ylabel("|X[k]|")
112         plt.grid(True)
113         plt.savefig("result/spectrum_original.png")
114         plt.close()
115         print("已輸出頻譜圖 result/spectrum_original.png")
116     # 理想濾波
117     Y_low = ideal_lowpass(X, cutoff)
118     Y_high = ideal_highpass(X, cutoff)
119     Y_butter_low = butterworth_lowpass(X, cutoff, n=2)
120     Y_butter_high = butterworth_highpass(X, cutoff, n=2)
121     # 逆轉換回時域
122     y_low = IDFT(Y_low)
123     y_high = IDFT(Y_high)
124     y_butter_low = IDFT(Y_butter_low)
125     y_butter_high = IDFT(Y_butter_high)
126     # 合併結果
127     out_low[start:end] = y_low
128     out_high[start:end] = y_high
129
130 out_butter_low[start:end] = y_butter_low
131 out_butter_high[start:end] = y_butter_high
132 # Step(8)轉回 8bit unsigned (0~255)
133 def to_uint8(x):
134     return np.clip(x + 128, 0, 255).astype(np.uint8)
135 out_low = to_uint8(out_low)
136 out_high = to_uint8(out_high)
137 out_butter_low = to_uint8(out_butter_low)
138 out_butter_high = to_uint8(out_butter_high)
139 # Step(9)輸出音檔
140 def save_wav(name, data):
141     wav_out = wave.open(f"result/{name}", "wb")
142     wav_out.setparams(params)
143     wav_out.writeframes(data.tobytes())
144     wav_out.close()
145 save_wav("b_low.wav", out_low)
146 save_wav("b_high.wav", out_high)
147 save_wav("b_butter_low.wav", out_butter_low)
148 save_wav("b_butter_high.wav", out_butter_high)
149 print("已輸出四個音檔:")
150 print("result/b_low.wav -> 理想低通")
151 print("result/b_high.wav -> 理想高通")
152 print("result/b_butter_low.wav -> Butterworth 低通")
153 print("result/b_butter_high.wav -> Butterworth 高通")

```

(1) b_low.wav(理想低通 ILPF)

- 聽起來:聲音悶、有包耳感、高頻消失。
- 原因:高頻被完全截斷(矩形濾波器), 只剩低頻 → 音質變厚但模糊。
- 改進:將 cutoff 從 40 調高至 60~80, 即可減少悶感、保留更多細節

(2) b_high.wav(理想高通 IHPF)

- 聽起來:沙沙聲、破音、語音變尖銳。
- 原因:低頻(主體能量)被切掉, 只剩高頻與雜訊。
- 改進:可改用 Butterworth 高通, 平滑過渡, 或調整cutoff 值至 80~120。

(3) b_butter_low.wav(Butterworth 低通 BLPF)

- 聽起來:比理想低通自然, 但音量較小、有微雜音。
- 原因:Butterworth 低通平滑但衰減過早, 高頻被過多削弱。
- 改進:提高 cutoff 至 80~100, 或是在輸出前加音量補償: $y = y * 1.5$

(4) b_butter_high.wav(Butterworth 高通 BHPF)

- 聽起來:幾乎和原音一樣, 變化不明顯。
- 原因:cutoff=40 太低, 高通幾乎全通過 → 濾波效果不明顯。
- 改進:將 cutoff 改為 100~150, 或提高 Butterworth 階數 n=3。

參考來源說明:

HW1_DFT_Filter_2025.ppt(課堂作業說明投影片)

DFT 與 IDFT 定義公式(課堂筆記):

$$X[k] = \sum x[n] * e^{-(j2\pi kn/N)}$$

$$x[n] = (1/N) \sum X[k] * e^{(j2\pi kn/N)}$$

Python 官方 wave 模組文件:<https://docs.python.org/3/library/wave.html>

Butterworth 濾波器設計原理

$$H(u,v) = 1 / (1 + (D(u,v)/D0)^{(2n)}) \text{ (低通)}$$

$$H(u,v) = 1 / (1 + (D0/D(u,v))^{(2n)}) \text{ (高通)}$$

本程式所有DFT / IDFT與濾波運算皆為手寫 for 迴圈, 未使用 numpy.fft 或任何現成頻域函式。

協助與偵錯:程式邏輯以及撰寫皆由本人先行完成, 再使用ChatGPT協助檢查錯誤。

工作進度四_作業題目3:其他音檔測試+自創加分項目(20%)

- 音檔來源:<https://sound-effects.bbcrewind.co.uk/search?cat=Clocks>
- 程式說明:

程式以8bit單聲道音訊檔(Clocks Striking.wav)作為輸入, 進行分段離散傅立葉轉換(DFT)分析、頻域濾波與逆轉換(IDFT)重建。設計並實作五種濾波器:理想低通(ILPF)、理想高通(IHPF)、Butterworth 低通(BLPF)、Butterworth 高通(BHPF), 以及自創加分項目——Butterworth 帶通濾波器(BPF)。每一種濾波器均以自行撰寫的DFT / IDFT函式完成, 未使用任何現成的numpy.fft函式庫。

程式流程為:

- 讀取音檔(例如 Nello.wav、Clocks Striking.wav), 將取樣資料分段後轉至頻域。

(2) 根據不同的濾波器遮罩函數 $H[k]H[k]H[k]$ 進行頻率篩選與平滑。

(3) 透過 IDFT 還原為時域訊號並輸出多個.wav音檔。

● 加分部分:

這部分是額外加的第五種濾波器，用的是Butterworth帶通濾波器 (BPF)，它的原理是把低通和高通濾波器結合在一起，只保留中間那段頻率(也就是聲音主要的區)，把太低和太高的頻率雜訊去掉。程式裡寫了一個 `butterworth_bandpass()` 的函式，裡面先做出低通和高通的遮罩，再把它們相乘形成帶通效果，設定 `low_cut=40`、`high_cut=100`、`n=2`，補:(函式，裡面先建立低通遮罩 $H_{low}(k)$ 和高通遮罩 $H_{high}(k)$ ，再用相乘的方式 $H_{band}(k) = H_{high}(k) * H_{low}(k)$ 來形成帶通濾波效果)。
整個流程和前面幾個濾波器一樣，都是DFT -> 濾波 -> IDFT，最後會輸出 `b_butter_band.wav`。聽起來雖然還是有沙沙聲，但聲音比較清楚、音質也較自然一些。

```
dft_filter_clock.py > ...
1 # HW1 頻域的聲音處理 (20%)
2 # 題目: 對 Clocks Striking.wav 進行 DFT + Filtering -> IDFT
3 # 實作理想低通 (ILPF)、理想高通 (IHPF)、
4 # Butterworth 低通 (BLPF)、Butterworth 高通 (BHPF)、
5 # 自創加分項目: Butterworth 帶通 (BPF)
6 # 限制: 不得使用 numpy.fft，只能自行撰寫 DFT / IDFT
7 import wave
8 import numpy as np
9 import os
10 import matplotlib.pyplot as plt
11 # 離散傅立葉轉換 DFT
12 def DFT(x):
13     N = len(x)
14     X = []
15     for k in range(N):
16         real = 0.0
17         imag = 0.0
18         for n in range(N):
19             angle = 2 * np.pi * k * n / N
20             real += x[n] * np.cos(-angle)
21             imag += x[n] * np.sin(-angle)
22         X.append(complex(real, imag))
23     return np.array(X)
24 # 離散傅立葉逆轉換 IDFT
25 def IDFT(X):
26     N = len(X)
27     x = []
28     for n in range(N):
29         s = 0
30         for k in range(N):
31             angle = 2 * np.pi * k * n / N
32             s += X[k] * complex(np.cos(angle), np.sin(angle))
33         x.append((s / N).real)
34     return np.array(x)
35 # 濾波器設計 (理想 & Butterworth)
36 def ideal_lowpass(X, cutoff):
37     N = len(X)
38     H = np.zeros(N)
39     H[:cutoff] = 1
40     H[-cutoff:] = 1
41     return X * H
42 def ideal_highpass(X, cutoff):
43     N = len(X)
44     H = np.ones(N)
45     H[:cutoff] = 0
46     H[-cutoff:] = 0
47     return X * H
48 def butterworth_lowpass(X, cutoff, n=2):
49     N = len(X)
50     H = np.zeros(N)
51     for k in range(N):
52         D = abs(k - N / 2)
53         H[k] = 1 / (1 + (D / cutoff) ** (2 * n))
54     return X * H
```

終端機顯示的結果 + 結果測試

```
C:\Users\user\Desktop>python dft_filter.py
Python 3.8.5 Shell 已啟動。輸入 > 256 Hz, 9179 Hz
進度: 0 / 9179
進度: 1000 / 9179
進度: 2000 / 9179
進度: 3000 / 9179
進度: 4000 / 9179
進度: 5000 / 9179
進度: 6000 / 9179
進度: 7000 / 9179
進度: 8000 / 9179
進度: 9000 / 9179
進度: 10000 / 9179
進度: 11000 / 9179
進度: 12000 / 9179
進度: 13000 / 9179
進度: 14000 / 9179
進度: 15000 / 9179
進度: 16000 / 9179
進度: 17000 / 9179
進度: 18000 / 9179
進度: 19000 / 9179
進度: 20000 / 9179
進度: 21000 / 9179
進度: 22000 / 9179
進度: 23000 / 9179
進度: 24000 / 9179
進度: 25000 / 9179
進度: 26000 / 9179
進度: 27000 / 9179
進度: 28000 / 9179
進度: 29000 / 9179
進度: 30000 / 9179
進度: 31000 / 9179
進度: 32000 / 9179
進度: 33000 / 9179
進度: 34000 / 9179
進度: 35000 / 9179
進度: 36000 / 9179
進度: 37000 / 9179
進度: 38000 / 9179
進度: 39000 / 9179
進度: 40000 / 9179
進度: 41000 / 9179
進度: 42000 / 9179
進度: 43000 / 9179
進度: 44000 / 9179
進度: 45000 / 9179
進度: 46000 / 9179
進度: 47000 / 9179
進度: 48000 / 9179
進度: 49000 / 9179
進度: 50000 / 9179
進度: 51000 / 9179
進度: 52000 / 9179
進度: 53000 / 9179
進度: 54000 / 9179
進度: 55000 / 9179
進度: 56000 / 9179
進度: 57000 / 9179
進度: 58000 / 9179
進度: 59000 / 9179
進度: 60000 / 9179
進度: 61000 / 9179
進度: 62000 / 9179
進度: 63000 / 9179
進度: 64000 / 9179
進度: 65000 / 9179
進度: 66000 / 9179
進度: 67000 / 9179
進度: 68000 / 9179
進度: 69000 / 9179
進度: 70000 / 9179
進度: 71000 / 9179
進度: 72000 / 9179
進度: 73000 / 9179
進度: 74000 / 9179
進度: 75000 / 9179
進度: 76000 / 9179
進度: 77000 / 9179
進度: 78000 / 9179
進度: 79000 / 9179
進度: 80000 / 9179
進度: 81000 / 9179
進度: 82000 / 9179
進度: 83000 / 9179
進度: 84000 / 9179
進度: 85000 / 9179
進度: 86000 / 9179
進度: 87000 / 9179
進度: 88000 / 9179
進度: 89000 / 9179
進度: 90000 / 9179
進度: 91000 / 9179
[+] Clocks Striking.wav 濾波完成!
輸出五個音檔:
result/clock_low.wav -> 理想低通
result/clock_high.wav -> 理想高通
result/clock_butter_low.wav -> Butterworth 低通
result/clock_butter_high.wav -> Butterworth 高通
```

結果:

(1) `clock_low.wav` (理想低通 ILPF)

- 聽起來: 聲音悶、沙沙聲重，有包耳感，高頻消失。
- 原因: 高頻波完全被截斷(矩形濾波器造成振鈴現象)，聲音變厚但模糊。
- 改進建議: 屬正常現象，可略調 `cutoff(40->60)` 或改用Butterworth低通平滑化。

(2) `clock_high.wav` (理想高通 IHPF)

```

48 def butterworth_lowpass(X, cutoff, n=2):
49     H[k] = 1 / (1 + (D / cutoff) ** (2 * n))
50     return X * H
51
52 def butterworth_highpass(X, cutoff, n=2):
53     N = len(X)
54     H = np.zeros(N)
55     for k in range(N):
56         D = abs(k - N / 2)
57         if D == 0:
58             H[k] = 0
59         else:
60             H[k] = 1 / (1 + (cutoff / D) ** (2 * n))
61     return X * H
62
63 # 自動加分項目: Butterworth 帶通濾波器 (BPF)
64 def butterworth_bandpass(X, low_cut, high_cut, n=2):
65     N = len(X)
66     H = np.zeros(N)
67     for k in range(N):
68         D = abs(k - N / 2)
69         if D == 0:
70             H[k] = 0
71         else:
72             # 低通 + 高通 = 帶通
73             H_low = 1 / (1 + (D / high_cut) ** (2 * n))
74             H_high = 1 / (1 + (low_cut / D) ** (2 * n))
75             H[k] = H_low * H_high
76     return X * H
77
78 # 主程式開始
79 if not os.path.exists("result"):
80     os.makedirs("result")
81
82 # Step 1. 讀取 Clocks Striking.wav
83 wav_in = wave.open("Clocks Striking.wav", "rb")
84 params = wav_in.getparams()
85 frames = wav_in.readframes(wav_in.getnframes())
86 wav_in.close()
87
88 # Step 2. 轉成 -128~127
89 data = np.frombuffer(frames, dtype=np.int16)
90 data = data.astype(np.int16) - 128
91
92 # Step 3. 設定分段長度與截止頻率
93 N = 256
94 cutoff = 40
95 low_cut = 30
96 high_cut = 100
97 if len(data) % N != 0:
98     pad = N - (len(data) % N)
99     data = np.append(data, np.zeros(pad))
100 segments = len(data) // N
101 print("資料長度:", len(data), "每段 N =", N, "共", segments, "段")
102
103 # Step 4. 建立五個輸出空間
104 out_low = np.zeros_like(data, dtype=float)
105 out_high = np.zeros_like(data, dtype=float)
106 out_butter_low = np.zeros_like(data, dtype=float)
107 out_butter_high = np.zeros_like(data, dtype=float)
108
109 # Step 5. 分段處理
110 for s in range(segments):
111     start = s * N
112     end = start + N
113     seg = data[start:end]
114     if s % 100 == 0:
115         print("進度:", s, "%", segments)
116         X = DFT(seg)
117         # 各種濾波器
118         Y_low = ideal_lowpass(X, cutoff)
119         Y_high = ideal_highpass(X, cutoff)
120         Y_butter_low = butterworth_lowpass(X, cutoff)
121         Y_butter_high = butterworth_highpass(X, cutoff)
122         Y_band = butterworth_bandpass(X, low_cut, high_cut)
123         # IDFT 回原
124         out_low[start:end] = IDFT(Y_low)
125         out_high[start:end] = IDFT(Y_high)
126         out_butter_low[start:end] = IDFT(Y_butter_low)
127         out_butter_high[start:end] = IDFT(Y_butter_high)
128         out_band[start:end] = IDFT(Y_band)
129
130 # Step 6. 輸出 0~255 unsigned (0-255)
131 def to_uint8(x):
132     return np.clip(x + 128, 0, 255).astype(np.uint8)
133 out_low = to_uint8(out_low)
134 out_high = to_uint8(out_high)
135 out_butter_low = to_uint8(out_butter_low)
136 out_butter_high = to_uint8(out_butter_high)
137 out_band = to_uint8(out_band)
138
139 # Step 7. 寫出音檔
140 def save_wav(name, data):
141     wav_out = wave.open(f"result/{name}", "wb")
142     wav_out.setparams(params)
143     wav_out.writeframes(data.tobytes())
144     wav_out.close()
145
146 save_wav("clock_low.wav", out_low)
147 save_wav("clock_high.wav", out_high)
148 save_wav("clock_butter_low.wav", out_butter_low)
149 save_wav("clock_butter_high.wav", out_butter_high)
150 save_wav("clock_butter_band.wav", out_band)
151
152 print("🔔 Clocks Striking.wav 濾波完成!")
153 print("輸出五個音檔:")
154 print("result/clock_low.wav -> 理想低通")
155 print("result/clock_high.wav -> 理想高通")
156 print("result/clock_butter_low.wav -> Butterworth 低通")
157 print("result/clock_butter_high.wav -> Butterworth 高通")
158 print("result/clock_butter_band.wav -> 自動加分的Butterworth 帶通 (BPF)")
159

```

- 聽起來:高頻沙沙聲明顯, 語音尖銳刺耳。
- 原因:主要語音能量(低頻部分)被濾除, 僅剩雜訊與共鳴。
- 改進建議:理想高通邊界過硬屬預期結果, 可改為Butterworth高通以平滑過渡。

(3) clock_butter_low.wav(Butterworth 低通 BLPF)

- 聽起來:仍有輕微沙沙聲, 但比理想低通自然。
- 原因:低頻平滑保留但能量下降, 高頻緩慢衰減。
- 改進建議:cutoff 提高至80~100可改善音量損失;可額外加入音量補償 $y = y * 1.5$ 。

(4) clock_butter_high.wav(Butterworth 高通 BHPF)

- 聽起來:語音偏尖、有輕微破音但保留部分原音。
- 原因:cutoff偏低(40)導致高通效果不明顯。
- 改進建議:cutoff 提高至100~150, 或將Butterworth階數調高至 $n = 3$ 。

(5) clock_butter_band.wav(Butterworth 帶通 BPF, 自動加分項)

- 聽起來:沙沙聲仍明顯, 但音色最平衡、語音可辨識度最高。
- 原因:同時抑制低頻與高頻, 只保留中頻(聲音主要區域);但分段DFT邊界造成相位不連續, 故仍有噪聲。
- 改進建議:不需修改(作業已完成所有條件),
- 若為實際應用可使用Overlap-Add或FFT平滑拼接法改善音質。

● 參考來源說明:

HW1_DFT_Filter_2025.ppt(課堂作業說明簡報)

Python 官方文件:wave 模組說明

<https://docs.python.org/3/library/wave.html>

NumPy 官方文件:陣列與數學運算

<https://numpy.org/doc/stable/>

協助與偵錯:程式邏輯以及撰寫皆由本人先行完成, 再使用ChatGPT協助檢查錯誤。

心得:

這次作業我花了蠻多時間在理解DFT和濾波器的部分，原本以為只是把聲音轉成頻域再轉回來而已，但實際做之後才發現每個步驟都要自己算、還要注意分段、角度、型態轉換等等，雖然有輸出的音檔聽起來有沙沙聲，不是完全乾淨的，但我覺得可以明顯聽出濾波效果的差別，像低通會變悶、高通會變尖、帶通比較自然，整體做下來真的讓我比較能理解課本裡講的頻域濾波原理，也學到怎麼用程式實際去驗證，更是學會如何實際去操作了。